

Article

A Fully Controllable UAV Using Curriculum Learning and Goal-Conditioned Reinforcement Learning: From Straight Forward to Round Trip Missions

Hyeonmin Kim ¹, Jongkwan Choi ¹, Hyungrok Do ² and Gyeong Taek Lee ^{3,*}¹ Department of Industrial Engineering, Yonsei University, Seoul 03722, Republic of Korea; rlagusals50@yonsei.ac.kr (H.K.); jk-choi@yonsei.ac.kr (J.C.)² Department of Population Health, NYU Grossman School of Medicine, New York, NY 10016, USA; doh03@nyu.edu³ College of Engineering, Gachon University, Global Campus, Seongnam 13120, Republic of Korea

* Correspondence: leegt@gachon.ac.kr

Abstract: The focus of unmanned aerial vehicle (UAV) path planning includes challenging tasks such as obstacle avoidance and efficient target reaching in complex environments. Building upon these fundamental challenges, an additional need exists for agents that can handle diverse missions like round-trip navigation without requiring retraining for each specific task. In our study, we present a path planning method using reinforcement learning (RL) for a fully controllable UAV agent. We combine goal-conditioned RL and curriculum learning to enable agents to progressively master increasingly complex missions, from single-target reaching to round-trip navigation. Our experimental results demonstrate that the trained agent successfully completed 95% of simple target-reaching tasks and 70% of complex round-trip missions. The agent maintained stable performance even with multiple subgoals, achieving over 75% success rate in three-subgoal missions, indicating strong potential for practical applications in UAV path planning.

Keywords: unmanned aerial vehicle; fully controllable UAV; path planning; goal-conditioned RL; curriculum learning



Academic Editor: Bo Li

Received: 25 November 2024

Revised: 27 December 2024

Accepted: 30 December 2024

Published: 31 December 2024

Citation: Kim, H.; Choi, J.; Do, H.; Lee, G.T. A Fully Controllable UAV Using Curriculum Learning and Goal-Conditioned Reinforcement Learning: From Straight Forward to Round Trip Missions. *Drones* **2025**, *9*, 26. <https://doi.org/10.3390/drones9010026>

Correction Statement: This article has been republished with a minor change. The change does not affect the scientific content of the article and further details are available within the backmatter of the website version of this article.

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Unmanned aerial vehicles (UAVs) have seen widespread utilization across several industries in recent years, including deliveries, search and rescue, surveillance, and military activities [1,2]. As UAVs' safety, maneuverability, and intelligence have advanced, their potential applications have broadened. To fully utilize the advantages of UAVs that operate independently without requiring human pilots, researchers must guarantee the performance of fully autonomous UAVs that can perform tasks themselves. In early research, military operations were mainly about UAV research and were developed to track the enemy well. Initially, UAVs are trained to chase the opponent UAV with rule-based techniques. However, due to the limitations of rule-based methods, UAVs can only carry out preprogrammed actions depending on specific circumstances. Consequently, it becomes challenging for UAVs to respond effectively in situations that fall beyond the scope of the established rules. Hence, recent studies have focused on reinforcement learning (RL) techniques rather than rule-based learning [3,4].

Nowadays, the missions of autonomous UAVs are extended to other fields and are generally divided into two categories: obstacle avoidance and path planning. Recently, numerous optimization techniques have been suggested to tackle the challenge of path

planning for UAVs. Conventional optimization techniques, such as A* [5], genetic optimization [6], and swarm optimization [7], have been mainly used for path planning. These approaches have proven to be effective in simple environments. However, it is reasonable to assume that there are numerous unexpected situations in a real-world environment. As a result, new path planning methods are needed, which are more effective in unknown and complicated environments.

Since the introduction of deep neural network (DNN), RL has evolved into deep reinforcement learning (DRL) through the integration of deep learning architectures. While traditional RL uses tabular methods or simple function approximators, DRL leverages DNNs to handle high-dimensional state spaces and learn complex feature representations directly from raw input data. This advancement has enabled RL to tackle more challenging problems in robotics and control [8–11]. Recent studies have shown remarkable progress in applying DRL to UAV control, particularly in handling wind disturbances [12] and achieving reliable tracking control [13,14]. DNNs excel in acquiring high-level features, and this property leads agents to learn the desired policy in high dimensional state space. These techniques offer the benefit of using the agent's varied experiences to choose the most effective strategy in an uncertain environment. As a result, recent DRL-based methods have played a leading role in the progress of autonomous path planning [15–17].

Traditional approaches utilizing DRL-based methods still have significant limitations in performing complex tasks. Although recent advances in goal-conditioned hierarchical RL show promise [18], and optimization techniques have been proposed for mission completion [19], several key challenges remain unresolved. Recent works have made progress in specific aspects. Ma et al. [12] demonstrated adaptive control under wind disturbances, and Wang et al. [14] achieved reliable tracking through distributional RL. However, the agent can still perform only specific tasks according to the predetermined goal.

Current methods face significant limitations in learning various maneuvers. Agents can easily learn the policy to carry out particular maneuvers, such as arriving at the target from a starting point. However, agents learn only how to move forward to reach the target in the shortest distance, making it challenging to learn various maneuvers such as rotation, climb, and descent. Yang et al. [20] attempted to address this through improved goal-conditioned representations, but the fundamental challenge of learning diverse behaviors remains.

The problem becomes more acute when considering complex mission profiles. For example, suppose that the UAV is trained to perform a maneuver that turns left. The agent cannot perform any task that does not involve turning left because the agent is not trained on how to perform other maneuvers. While methods like those proposed by Zhao et al. [21] have improved multi-goal learning, they still struggle with truly diverse action spaces. It is hard to train the UAV to perform various tasks the user desires by conventional training methods. The user must define new formulations and train the agent for each task.

Furthermore, significant challenges arise when the UAV has to learn the path planning policy to return to its starting point. For fixed-wing UAVs that cannot perform vertical takeoff and landing (VTOL), rotation after reaching the target point is necessary for return navigation. This requires a fundamentally different set of skills from forward navigation. While recent work by Nasiriany et al. [22] has improved planning with goal-conditioned policies, the challenge of seamlessly combining forward and return navigation remains particularly difficult for an agent who has only learned how to proceed.

Recent work has demonstrated the effectiveness of efficient state representations for UAV control in single-objective tasks [1]. However, handling complex missions with multiple objectives requires extending beyond these existing approaches. This study presents an integrated approach to UAV path planning, demonstrated in Figure 1 where multiple

subgoals (blue points) guide the UAV through complex trajectories while maintaining controllability. We propose a fully controllable UAV path planning technique that allows agents to perform a variety of missions without the requirement for task-specific adjustments. It is very challenging for a UAV agent to perform various missions while being given a destination in real-time from its starting point. Our research aims to create a fully controllable UAV agent that can reach any goal where it exists in real-time, including both outbound and return navigation capabilities. The main contributions of this paper are as follows:

1. **Novel Integration Methodology:** We propose a unique combination of curriculum learning and goal-conditioned RL that enables fully autonomous UAV control in ways not previously demonstrated. Unlike existing methods that typically focus on either curriculum learning or goal-conditioning separately, our approach synergistically combines both to achieve superior performance in complex missions.
2. **Advanced Mission Handling Framework:** Our approach successfully handles round-trip missions and complex directional changes, capabilities that existing approaches have struggled to achieve. The method demonstrates particular effectiveness in scenarios that require multiple waypoint navigation and return-to-base operations, significantly expanding current UAV operational capabilities.
3. **Comprehensive Empirical Validation:** Through extensive experiments, we demonstrate that our approach achieves significantly higher success rates (>70% for complex missions) compared to baseline methods (<35% for similar scenarios). Our experimental results validate the effectiveness of progressive learning in enhancing UAV control capabilities.

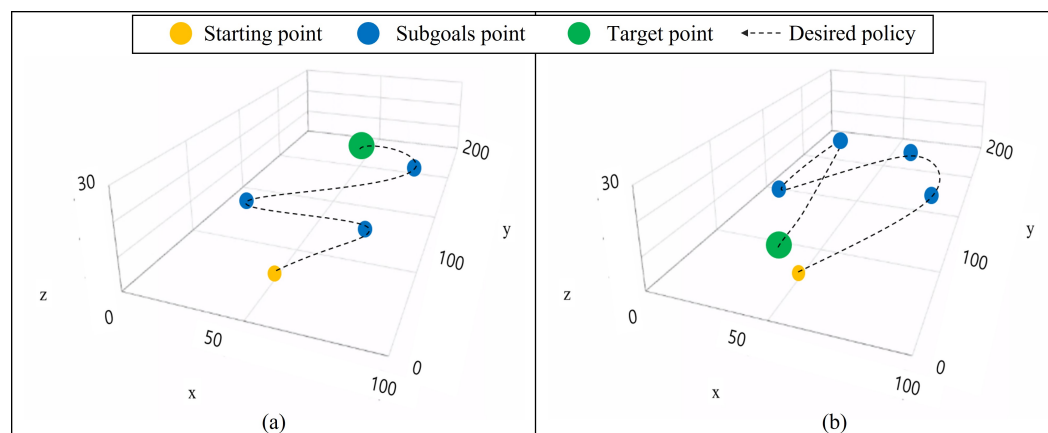


Figure 1. The experiment setup for UAV path planning with subgoals. Both scenarios demonstrate how subgoals guide the UAV to accomplish complex missions. (a) Real-time path planning with multiple subgoals where the UAV must reach the target point by passing through intermediate subgoals. (b) Round-trip mission scenario where the UAV must return to a point near its starting location after reaching all subgoals.

Goal-conditioned RL allows an agent to accomplish various goals under specific conditions. Unlike traditional RL solutions, it combines observation with an additional goal for the agent to make decisions that match various goals. Goal-conditioned RL is mainly applied to robotics and has recently performed well in various fields, including path planning problems [20,22,23]. Goal-conditioned RL for the planning of routes uses subgoals as milestones that form part of the agent's trajectory. The agent employs subgoals to progress towards the final target. However, in early research, there is a constraint that the agent cannot accomplish complex missions, such as returning to its starting position after departure.

We utilize curriculum learning and goal-conditioned RL to accomplish these objectives. Curriculum learning implements a progressive training approach where complex behaviors are built upon simpler foundational skills. This methodology enables the systematic development of advanced capabilities through incremental learning stages. Through this structured approach, we propose a learning method that allows the agent to master basic tasks before progressing to more complex missions. The combination of curriculum learning with goal-conditioned RL enables our agent to first master simple subgoal-reaching maneuvers before advancing to more complex behaviors, such as return navigation.

The rest of the paper is organized in the following ways: Section 2 provides preliminaries of the study. Section 3 outlines the UAV's dynamic model and introduces UAV training and testing simulation environments. The methodology for the real-time path planning is outlined in Section 4. Section 5 includes a series of experiments, whereas Section 6 summarizes the article and shows our contributions.

2. Background

2.1. Goal-Conditioned RL

This section utilizes Markov Decision Process (MDP) to express the environment. The MDP is written as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{T}$ is a state transition probability matrix, R is a reward function, and γ is a discount factor.

The agent performs tasks according to a policy (π) that determines the probability distribution of selecting actions given the current state: $\pi(\mathbf{a}|\mathbf{s}) = P[\mathbf{S}_t = \mathbf{s}, \mathbf{A}_t = \mathbf{a}]$. Using the state value function $v_\pi(\mathbf{s})$ and state-action value function $Q_\pi(\mathbf{s}, \mathbf{a})$, the agent learns to maximize the expected cumulative reward:

$$J(\pi) = \mathbb{E}_{\substack{\mathbf{a}_t \sim \pi(\cdot|\mathbf{s}_t), \\ \mathbf{s}_{t+1} \sim \mathcal{T}(\cdot|\mathbf{s}_t, \mathbf{a}_t)}} \left[\sum_t \gamma^t R(\mathbf{s}_t, \mathbf{a}_t) \right]. \quad (1)$$

Goal-conditioned RL extends this framework by adding a goal set to the MDP formulation, represented as $(\mathcal{S}, \mathcal{G}, \mathcal{A}, \mathcal{T}, R, \gamma)$, where $\mathcal{G}$ refers to the collection of goals including subgoals. This modification allows the agent to learn policies for multiple goals, maximizing the expected reward as:

$$J(\pi) = \mathbb{E}_{\substack{\mathbf{a}_t \sim \pi(\cdot|\mathbf{s}_t, \mathbf{g}), \\ \mathbf{g} \sim \mathcal{G}, \\ \mathbf{s}_{t+1} \sim \mathcal{T}(\cdot|\mathbf{s}_t, \mathbf{a}_t)}} \left[\sum_t \gamma^t R(\mathbf{s}_t, \mathbf{a}_t, \mathbf{g}) \right]. \quad (2)$$

2.2. Actor-Critic RL

Actor-critic is a policy-based RL algorithm that combines value-based and policy-based approaches through two network components [24]. Value-based RL selects actions based on Q-value or value, while policy-based RL directly learns a policy through probability distributions, making it suitable for high-dimensional and continuous tasks.

In an actor network, the probability distribution of an action, $\pi(\mathbf{a}|\mathbf{s})$, can be represented by the policy π , where the current action and state are included in $\mathbf{a} \in \mathcal{A}$ and $\mathbf{s} \in \mathcal{S}$, respectively. The actor aims to maximize the state-action value function $Q_\pi(\mathbf{s}, \mathbf{a})$ by determining a policy π , which can be represented as follows:

$$\begin{aligned} J(\pi) &= \mathbb{E}_\pi[Q_\pi(\mathbf{s}, \mathbf{a})] \\ &= \int_{\mathcal{S}} p^\pi(\mathbf{s}) \int_{\mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) Q_\pi(\mathbf{s}, \mathbf{a}) d\mathbf{a} d\mathbf{s}. \end{aligned} \quad (3)$$

where $p^\pi(\mathbf{s})$ is the distribution of state. The role of the critic network is to evaluate the value of the current state and employ it to train the actor network. Minimizing the square of the TD error is the critic's goal, as follows:

$$L(\pi) = [(r + Q_\pi(\mathbf{s}', \mathbf{a}') - Q_\pi(\mathbf{s}, \mathbf{a}))]^2. \quad (4)$$

2.3. Random Network Distillation

In RL, exploration is essential for efficient learning [25,26]. Random network distillation (RND) [27] is a popular exploration strategy that uses the prediction error between two networks as an exploration bonus. It consists of a fixed randomly initialized target network f and a predictor network $\hat{f}$ trained to match the target's outputs. The predictor network, parameterized by $\theta_{\hat{f}}$, minimizes the mean squared error between its predictions $\hat{f}(\mathbf{x}; \theta)$ and the target network's outputs $f(\mathbf{x})$.

The prediction error serves as an exploration bonus, being larger for novel states and smaller for frequently visited ones. This naturally encourages exploration of unfamiliar states while gradually reducing the incentive to revisit well-explored areas. As the agent explores new regions of the state space, it receives higher bonuses, promoting systematic exploration of the environment.

2.4. Self-Imitation Learning

Self-imitation learning (SIL) is an algorithm that operates within the on-policy RL framework [28], where agents learn optimal policies through direct environmental interaction. The agent explores the environment under its current policy and updates it based on obtained rewards.

SIL enhances learning by replicating valuable past behaviors stored in a replay memory [29]. Specifically, it leverages successful past experiences when their returns exceed current value estimates, while avoiding the exploitation of suboptimal transitions. This approach has shown effectiveness when implemented with on-policy RL algorithms [28,30]. The SIL loss is formulated as:

$$\begin{aligned} \mathcal{L} &= \mathbb{E}_{\mathbf{s}, \mathbf{a}, R \in \mathcal{D}} [\mathcal{L}_{\text{policy}} + \beta \mathcal{L}_{\text{value}}], \\ \mathcal{L}_{\text{value}} &= \frac{1}{2} \|(R - V_\theta(\mathbf{S}))_+\|^2, \\ \mathcal{L}_{\text{policy}} &= -\log \pi_\theta(\mathbf{a}|\mathbf{s})(R - V_\theta(\mathbf{S}))_+, \end{aligned} \quad (5)$$

where the value network, $V_\theta(\mathbf{s})$, and the policy network, π_θ , are respectively parameterized by θ and $(\cdot)_+ = \max(\cdot, 0)$. $\beta \in \mathbb{R}^+$ is utilized to control the value loss. The $(\cdot)_+$ operator is employed in the loss function for training the policy and value network. It is specifically used for transitions in which the current value surpasses the previous return.

2.5. Curriculum Learning

First, curriculum learning is proposed to train deep learning models from easy to complex data [31]. Curriculum learning is quite logical since it is most likely similar to how humans learn. Furthermore, curriculum learning has developed to adjust the difficulty through the task and model perspective. This method has sound effects, such as improving the speed of convergence of deep learning models and reducing the tendency to get stuck in local minima.

Curriculum learning has been applied in diverse domains, including natural language and robotics [32,33]. Recently, research has continued to apply curriculum learning to RL [34]. Curriculum learning is mainly utilized as a training method for accumulating the agent's experience to maximize performance or training efficiency for final tasks. In other

words, it is a method that transfers accumulated agent knowledge by allowing the agent to perform simple tasks to acquire complicated tasks more efficiently and quickly.

3. Problem Definition

3.1. Environment

The environment is a region of $100 \times 200 \times 30$ km in height, width, and depth. This environment aims to train UAVs to achieve the objective within a specified time limitation. In this paper, We conducted experiments in a simulation environment with 3 degrees of freedom (3 DoF) with a continuous state space as follows [35]:

$$\begin{aligned}\dot{V} &= \frac{T - D}{m} - g \sin \gamma, \\ \dot{\psi} &= \frac{gn \sin \phi}{V \cos \gamma}, \\ \dot{\gamma} &= \frac{g}{V(n \cos \phi - \cos \gamma)}, \\ \dot{x} &= V \cos \gamma \cos \psi, \\ \dot{y} &= V \cos \gamma \sin \psi, \\ \dot{z} &= V \sin \gamma.\end{aligned}\tag{6}$$

The coordinates of the UAV in the environment are represented by (x, y, z) , while the velocity is denoted by V . The heading angle is represented by ψ , and the pitch angle is represented by γ . The symbol g represents the acceleration due to gravity, D represents air resistance, and m represents mass. The inputs for the UAV's controls are denoted by T , n , and ϕ , respectively, representing the engine thrust, load factor, and bank angle.

It is important to note that this 3 DoF model is a simplified representation that does not account for several critical aerodynamic factors. The model omits lift forces and stall conditions, which are crucial elements in real aircraft dynamics. This simplification means that the model may allow physically unrealistic maneuvers, particularly in situations involving extreme angles or rapid velocity changes. The absence of these aerodynamic constraints could result in trajectories that would be impossible for actual fixed-wing aircraft to execute.

In this environment, we consider realistic flight dynamics while maintaining computational tractability. The state space is continuous, requiring discretization for practical implementation. As illustrated in Figure 2, changes in engine thrust impact the UAV's velocity, consequently influencing all angles and positions. Furthermore, the bank angle and load factor determine the heading and pitch angles.

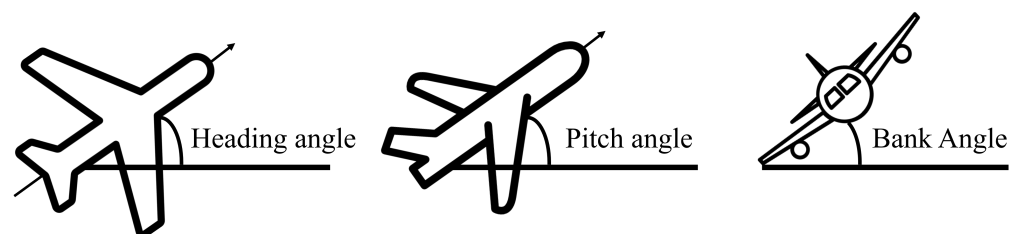


Figure 2. A graphic depicting the heading, pitch, and bank angles. These elements define the UAV's next position.

3.2. Test Environment

This study aims to guide the UAV toward its goal by sequentially achieving subgoals. Examples of a test scenario are shown in Figure 1. The UAV's goal continuously changes

after it leaves the subgoal. Rather than focusing on reaching a predetermined target point, this work aims to allow a single UAV agent to execute a diverse range of sub-tasks.

3.3. State

In RL, the state vector must contain appropriate information to enable the agent to respond to the environment adequately. A state vector must include information related to the UAV's coordinates and angles and the target it aims to achieve. However, it is challenging to represent information such as coordinates and angles of the UAV. For instance, describing the agent's coordinates with a one-hot encoding vector requires more dimensions as the environment becomes more complicated and can only represent integer coordinates. Furthermore, representing angles in a state representation for RL is challenging due to their continuous rotation over an entire 360-degree period.

An efficient coordinate vector (ECV) technique addresses these challenges, as explained in [1]. The ECV method uses a one-hot encoding vector to first encode each axis' coordinates before combining them to represent coordinates. Moreover, angles are defined using a polar coordinate system. ECV method transforms angles into positions on the upper part of a circle. Through the use of ECV, the state vector can be effectively represented as follows:

- The UAV's angle and distance relative to the goal
- The UAV's Heading, pitch, and bank angle

The state vector excludes the absolute coordinates of the UAV agent. The agent can only learn how to complete a specific task in a specific location if the UAV's absolute coordinate is contained to the state vector. Thus, we construct a state vector by only including angles and the relative coordinates concerning the relation of the UAV with the target point. The state vector is specified as a column vector consisting of two states, denoted as $[\mathbf{S}_{UAV}, \mathbf{S}_{goal}]^T$. The state vector $\mathbf{S}_{UAV}$ contains the information on the present position of the UAV. The state vector is specified as a column vector consisting of two states, denoted as $[\mathbf{S}_{UAV}, \mathbf{S}_{goal}]^T$. The state vector $\mathbf{S}_{UAV}$ contains the information on the present position of the UAV. The state vector $\mathbf{S}_{goal}$ denotes the relative position of the UAV with the desired target. The ECV can be used to represent each state vector in the following manner:

$$\begin{aligned}\mathbf{S}_{UAV} &= [\delta(\phi), \delta(\psi), \delta(\gamma)]^T, \\ \mathbf{S}_{goal} &= [\delta(\text{dist}(\mathbf{p}_{UAV}, \mathbf{p}_{goal})), \delta(\text{angle}(\mathbf{p}_{UAV}, \mathbf{p}_{goal}))]^T,\end{aligned}\quad (7)$$

where δ denotes the function that implements the ECV approach. $\mathbf{p}_{UAV}$ and $\mathbf{p}_{goal}$ indicate the UAV's and the goal's positions, respectively. Additionally, $\text{dist}(\mathbf{p}_{UAV}, \mathbf{p}_{goal})$ and $\text{angle}(\mathbf{p}_{UAV}, \mathbf{p}_{goal})$ indicate the angle and distance between the target and the UAV's current coordinates.

3.4. Action

To guarantee a realistic behavior for the UAV, the 3 DoF mass model is determined for the RL agent's control inputs (load factor, bank angle, and engine thrust). Each input can be controlled in three ways: increase, hold, or decrease. Consequently, The RL agent is capable of performing a total of 27 different actions, and the area for action is set up as follows:

$$\mathcal{A} = \{(\mathbf{a}_T, \mathbf{a}_\phi, \mathbf{a}_n) | \mathbf{a}_T, \mathbf{a}_\phi, \mathbf{a}_n \in \{\text{increase, hold, decrease}\}\}, \quad (8)$$

where $\mathbf{a}_n$, $\mathbf{a}_\phi$, and $\mathbf{a}_T$ specify the maneuvers for adjusting the load factor, bank angle, and engine thrust, accordingly.

While this discrete action space makes the learning problem tractable, it introduces certain limitations. The restricted set of actions may not capture the full range of subtle

control adjustments that would be possible in continuous control. This could result in less smooth trajectories and potentially suboptimal maneuvers, particularly in scenarios requiring fine-grained control. Furthermore, the discretization of control inputs may not allow for the precise adjustments often needed in real-world flight scenarios.

3.5. Reward

The agents receive a reward after each episode. Agents can obtain three types of rewards. The rewards are established as follows:

- Upon successfully reaching the final target (+20)
- Upon departing from its environment (−10)
- When the fuel is depleted before reaching the goal (−5)

We also penalize -0.01 for every time step to prevent the agent from wandering around instead of reaching the goal directly. During training with goal-conditioned RL, this same reward structure is applied whether the current target is a subgoal or the final goal. In this approach, each subgoal is treated as a temporary goal state during training. The agent's state representation includes relative position information to the current subgoal, allowing it to learn policies for reaching intermediate points. When a subgoal is reached, the next subgoal becomes the temporary target while maintaining the same reward structure. The mission is considered successful only when the agent passes through all subgoals and reaches the final goal, though the episode may terminate earlier if fuel is depleted or the agent departs from the environment.

4. A Fully Controllable UAV in the Path Planning

In this section, we describe an approach for controlling the UAV to various locations to perform real-time path planning. The primary aim of this study is to enable the agent to arrive at any desired spot, regardless of location. It is assumed that the agent can arrive at any point if it can be guided by subgoals that are set to encourage reaching the final goal. In earlier studies, there were limitations to the number of goals an agent could reach or tasks it could perform. However, our suggested approach seeks to enable the agent to accomplish the goal via multiple subgoals, wherever subgoals exist. There is one recent study on reaching the target point, which exists in front of the UAVs via subgoals, as shown in Figure 3a [36]. However, there have been no studies that maneuver to return to the final target near the starting point by bypassing subgoals. To achieve fully controllable UAVs, we combine goal-conditioned RL and curriculum learning. The agent in goal-conditioned RL can accomplish its final goal through subgoals. We intend to enhance the learning efficiency of RL by implementing goal-conditioned RL. This approach enables the agent to implement the targeted policy under the given environment. When the agent acquires knowledge of the subgoals, it can be guided to accomplish the final goal by carrying out those subgoals. Consequently, the UAV agent can successfully attain the goal by reaching many user-defined sub-points.

Figure 3a illustrates the abilities the UAV agent can be inherent with goal-conditioned RL. Typically, in classic RL, the agent is limited to learning a single route from the beginning point to point C, as depicted in Figure 3c. The UAV agent enables to reach two sub-routes within a single route by employing goal-conditioned RL. However, as depicted in Figure 3b, the UAV agent cannot execute intricate sub-routes without curriculum learning.

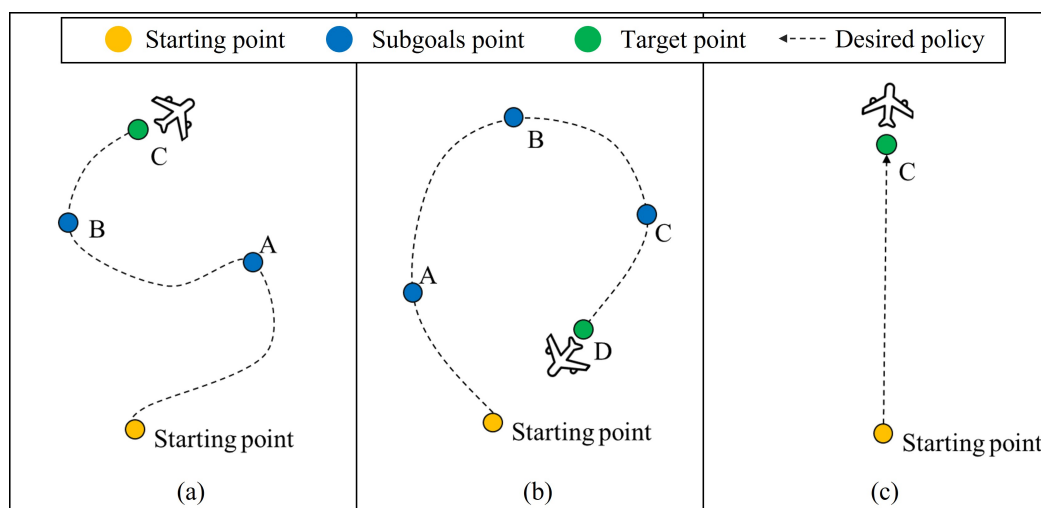


Figure 3. The examples of path planning that utilize goal-conditioned RL. (a) The agent goes forward to the target point with goal-conditioned RL. (b) The agent performs a round-trip to the target point with our proposed method. (c) With conventional RL, the agent is taught to arrive at the goal in the shortest distance and does not learn to perform zig-zag maneuvers.

4.1. Learning the Basic Flight for the Agent

Training a UAV agent to perform complex maneuvers such as those shown in Figure 4b directly from scratch is challenging and often fails to converge. As demonstrated in Figure 4a, the agent needs to first master basic flight skills through progressively more challenging tasks. Through curriculum learning, the agent can systematically build up its capabilities, starting with simple straight-line flights and gradually advancing to more complex maneuvers. This progressive approach encourages the agent to develop robust policies step by step, ultimately enabling it to achieve complex mission objectives like those shown in Figure 4b.

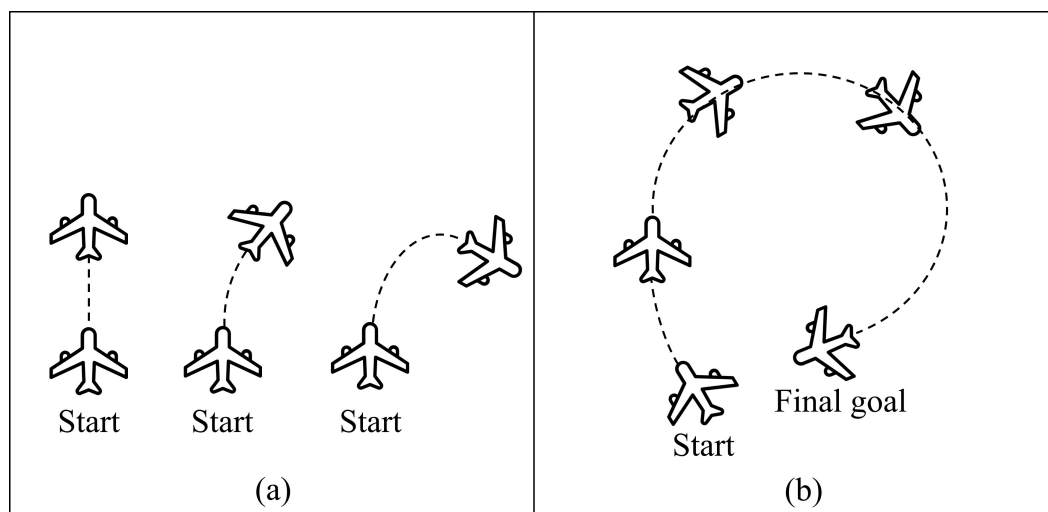


Figure 4. Progressive flight training through curriculum learning. (a) Step-by-step training of fundamental flight maneuvers with increasing complexity. (b) Final target mission profile requiring complex navigation capabilities.

The main purpose of the first stage is to instruct the UAV agent to achieve the targets close to the initial location. As shown in Figure 5a, the target point is chosen randomly from among goal candidates for each episode. There are two rules for establishing goal candidates. First, from the starting point, the candidates for the goal must be spread out, not just at the top and bottom but also on the left and right. Figure 5b shows an example

of a random noise assigned to a goal at training time. The coordinates of the goal are given a random noise to vary the goal's location for each episode. In common, random noise is added to every goal candidate's x-axis, y-axis, and altitude. If the agent is trained through the fixed goal coordinates, it may learn the simple maneuvers to reach specific coordinates. Thus, random noise is given to the goal coordinates so the agent can learn robust maneuvers. It enables the agent to acquire the ability to navigate in all directions. As a result, the agent learns basic maneuvers such as climbing, descending, and rotating. Second, the goal must be near the initial place. The agent will be able to move forward at this stage if we train it with the remote target. If the goal is situated close to the right of the starting point, this will encourage the agent to turn to the right. Similarly, the agent will learn a variety of basic maneuvers with close targets. Consequently, 6 goal candidates are chosen to train the agent following the rules, three on the left and three on the right relative to the agent.

The UAV agent can employ subgoals to get to the target through the first stage utilizing goal-conditioned RL. Suppose the agent needs to reach point C, as shown in Figure 3a. The transitions split into several segments with subgoals as the UAV agent nears the final goal. Then, the agent transits to the nearest subgoal and is guided to reach the final target by completing these intermediate missions. Consequently, the agent can carry out the various maneuvers with subgoals and complete tasks in front of the agent.

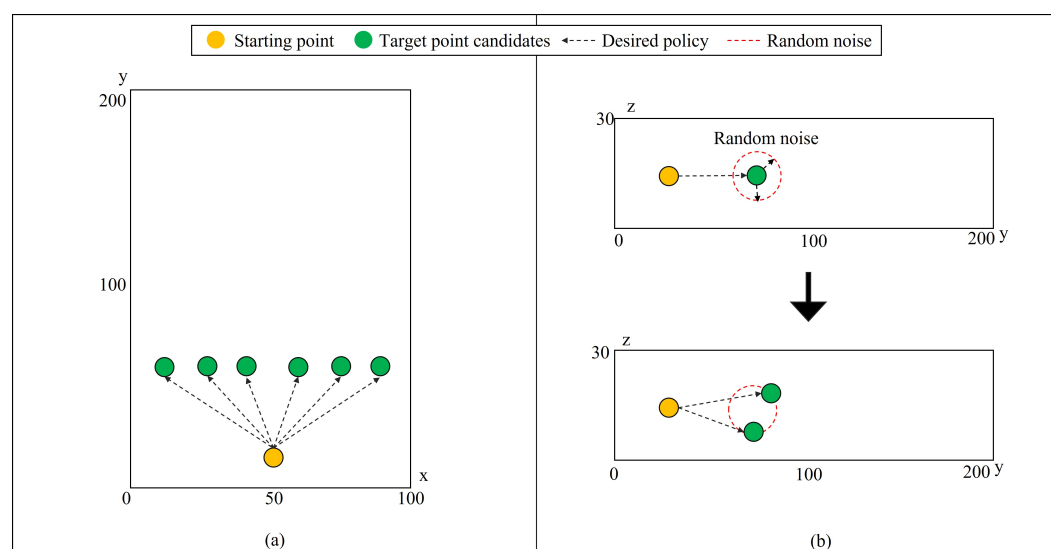


Figure 5. The example of goal candidates of the UAV in the first stage. (a) Top view of the goal candidates. (b) Side view of the goal candidates with an example of random noise.

4.2. Learning Various Flight Maneuvers

The main purpose of the second stage is to enhance the navigating abilities of the UAV agent through curriculum learning and goal-conditioned RL. An agent who has completed training in the first stage must perform the missions, gradually increasing the difficulty. Figure 6 shows the overall learning process of the second stage. We formulate the training process in three steps. First, the goal is assigned as in the previous training process. If the agent reaches there, the new goal is given around the first goal so that the agent reaches the two goals in succession. For example, Figure 6b shows the process of the first step of the second stage. Repeating these steps twice encourages the agent to reach four consecutive goals in the final step. In this stage, subgoals are generated in the agent's path, and the agent learns them through goal-conditioned RL, as in the previous stage. The earlier goals are considered subgoals, and the agent is trained to reach a more challenging final goal via subgoals. In other words, it learns in this stage by utilizing goal-conditional RL within

curriculum learning. Finally, the agent's capabilities are strengthened by performing tasks step by step.

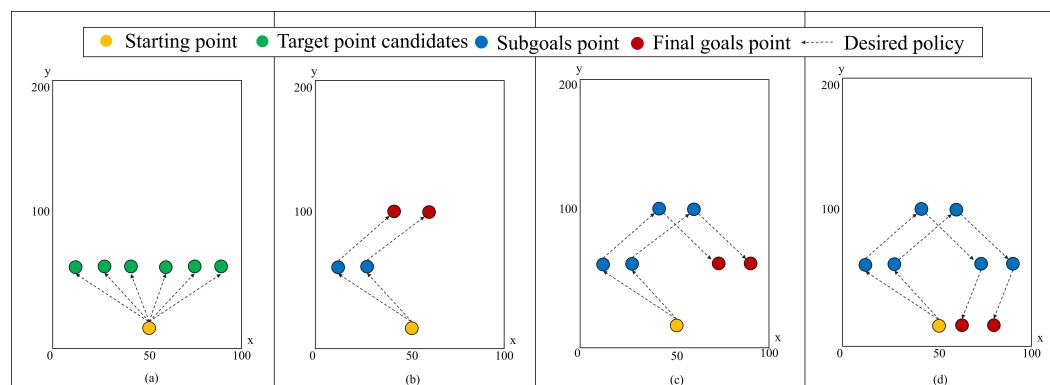


Figure 6. The Examples of learning the UAV in the second stage with curriculum learning. (a) The UAV agent is trained in the first stage, (b) The first step of the second stage, (c) The second step of the second stage, (d) The last step of the second stage.

In this stage, the goal points are set to allow the agent's progressive return to the beginning point. This method will enhance the agent's rotational ability compared to the previous step. In the first stage, the agent has not learned how to turn more than 90 degrees, making it challenging to guide the agent back to the goal located near the beginning location. After the final phase, the agent can execute intricate movements like a round-trip. Eventually, regardless of the location of the goal, the agent can successfully achieve it by completing the subgoals. Moreover, even if the agents go to the same destination, they can bypass a variety of subtasks depending on the order and combination of subgoals.

5. Experiment

This section describes experimental validation of the proposed method. We evaluate our approach through comprehensive experiments and analyses. The experiments consist of both training and test phases. We first describe the model architecture and training configurations, followed by various test scenarios designed to evaluate the agent's performance.

5.1. Model Architecture

In the experiments, we wanted to tackle the challenging path planning problem with user-designed subgoals that were randomly generated and had yet to be seen by the agent during training. Due to the agent's training in a complex learning environment comprising continuous state and action spaces, we introduced on-policy actor-critic network [28,37], which is proper to handle such problems. To take advantage of the agent's advantageous past experiences, we applied the SIL [29] approach, accelerating the training process and enabling the agent to get the desired policy more quickly. Furthermore, We used RND to encourage the agent's exploration, which provides a bonus reward for exploration during the RL process [27]. The exploration bonus is provided to the agent to encourage ongoing exploration of new states. As a result, the final architecture of the RL model consists of an actor-critic network with the SIL and RND.

Figure 7a illustrates the structure of the policy network used in the actor-critic network, while Figure 7b demonstrates the architecture of the target and predictor network used in the RND. Except for the output layer, the architecture of the policy network is identical to the value network. The dimension of the output layer of the value network is 1, as it determines the state's value function. Moreover, the architecture of both the target and predictor networks is identical.

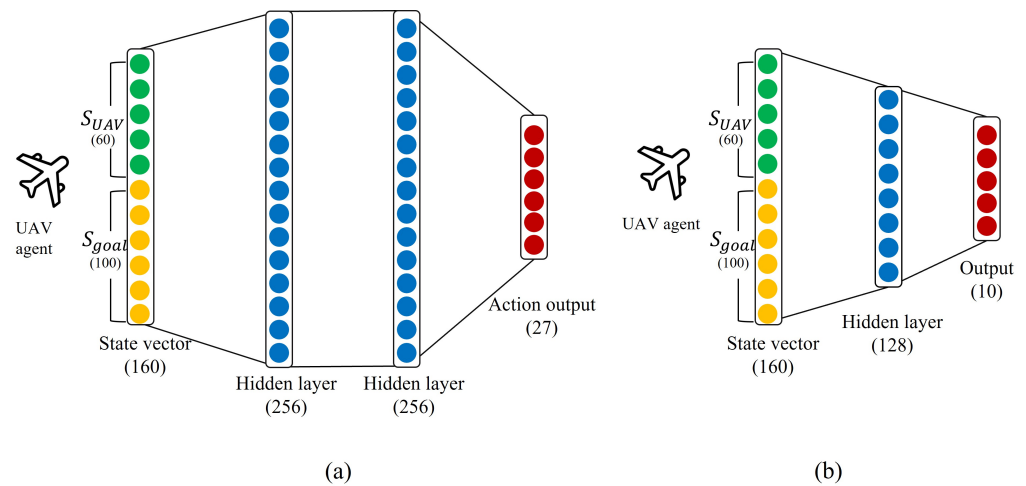


Figure 7. Network architectures. (a) policy network (b) target and prediction networks for RND.

5.2. Training Phase

We set different levels for the training and test phases to evaluate the agent's performance. In the training phase, we utilized a two-stage approach to train the agent for fully controllable UAVs. In addition, the training phase is divided into four stages, each with a different training setting. As depicted in Figure 6a, in the first stage, we defined the fixed initial point. Moreover, the agent selected the first target point from the available goal candidates arbitrarily. First, during the 100,000 training episodes, the agent has to finish to reach the goal. In the remaining stages, the agent received training to achieve the goal set, which includes the succession of increasingly tricky targets. The agent was trained to perform the tasks of each stage for 50,000, 30,000, and 20,000, respectively.

5.3. Test Phase

We set several scenarios during the test phase to validate the ability of the UAV agent to plan a real-time path. We devised three to four subgoals for each scenario and verified the agent's ability to achieve both the subgoals and the final goal. The agent does not know the information about the subgoals beforehand. When being provided with the subgoal while in flight, the agent updates its state to correspond with the present location of the UAV with the subgoal. The final goal was established under two conditions: significant distance from or close to the initial point, which the agent has not previously visited during the learning. In the first scenario, both the subgoal and the endpoint are located far from the agent's starting point. Figure 8 shows the three cases of the first setting.

We configured the different subgoal combinations in the second test setting as illustrated in Figure 9. We position the goal close to the initial point, making the agent perform a round-trip. Compared to the previous scenario, the UAV agent faces more difficulty completing tasks in this setting. In this test setting, we considered both clockwise and counter-clockwise orientations to validate the agent's ability to turn rather than simply rotating in one direction. We also conducted a test to validate missions that require directional changes, as shown in Figure 9c,d. Moreover, we compared the baseline agent without curriculum learning with our proposed method. The baseline agent is trained with just goal-conditioned RL. In the baseline, SIL and RND are applied in the training phase, the same as the proposed method.

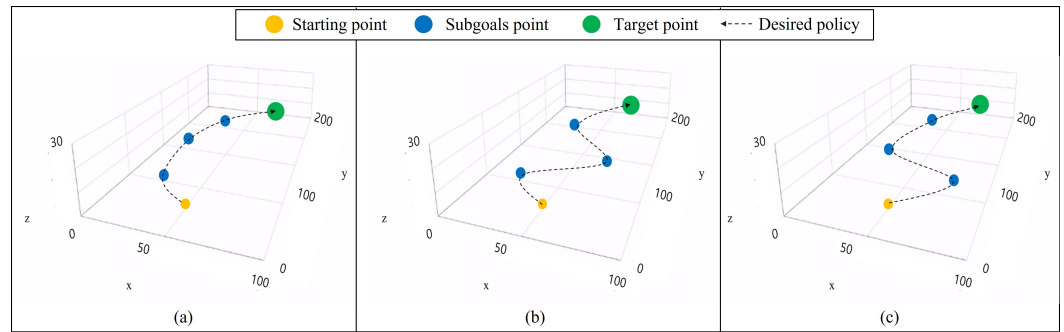


Figure 8. Three instances of the first scenario within the testing setting (a–c). After finishing training, the agent progresses to the final goal, bypassing every subgoal that they had never reached before.

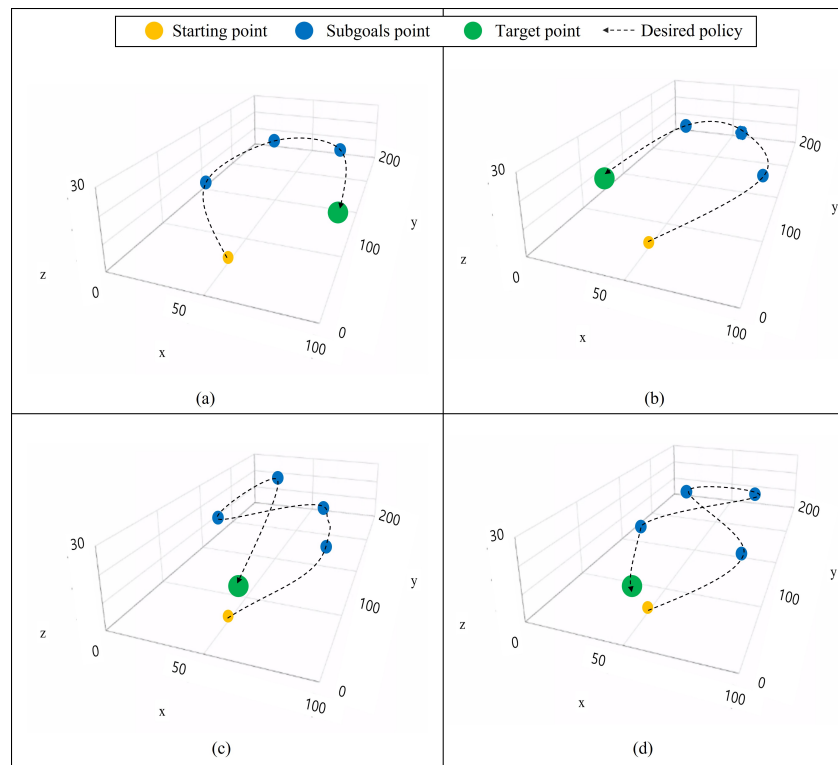


Figure 9. Four instances of the second scenario within the test setting (a–d). Through the first three or four subgoals, the agent arrives at the end goal. The final goal is more difficult than the first scenario.

6. Result

In this section, we present the experimental results and analysis of our proposed method. First, we examine the overall learning progression through both training phases, demonstrating how the agent's performance evolves during curriculum learning. Then, we evaluate the agent's performance across different test scenarios, from simple target-reaching tasks to complex round-trip missions. Finally, we provide a analysis comparing our method with baseline approach.

6.1. Overall Learning Progression

Figure 10a represents the training results of the first training stage. Our research revealed that the agent accomplished the goal as the episode proceeded. Achieving the goal is not difficult for the agent in this phase compared to the second stage. Nevertheless, because the goals for each episode are randomly chosen, it is difficult to fully converge on the learning curve. Occasionally, the agent failed to reach the intended goal. Nevertheless, as the episode count neared 100,000, the agent achieved the goal with an increased

likelihood. Because the structure of networks is not complicated, the number of training episodes determines the time spent training. The total time of our experiment was 4 h, during which we completed 100,000 episodes in the first training phase.

Figure 10b represents the training results of the second training stage. This stage consists of three steps, and the training for each step is performed consecutively. For example, when the training episode of the first step is over, the second step is performed immediately. Each step has a different difficulty level, and when the number of subgoals for the agent to reach increases, more rewards are needed to succeed. Moreover, because each episode goal was chosen randomly and given random noise to the goal coordinates, the learning curve failed to reach convergence ideally at the end of the step. Nevertheless, as the episode count neared its maximum (50,000, 30,000, and 20,000, respectively), the agent achieved the target with an increased likelihood. The second stage required 6 h to accomplish 100,000 episodes.

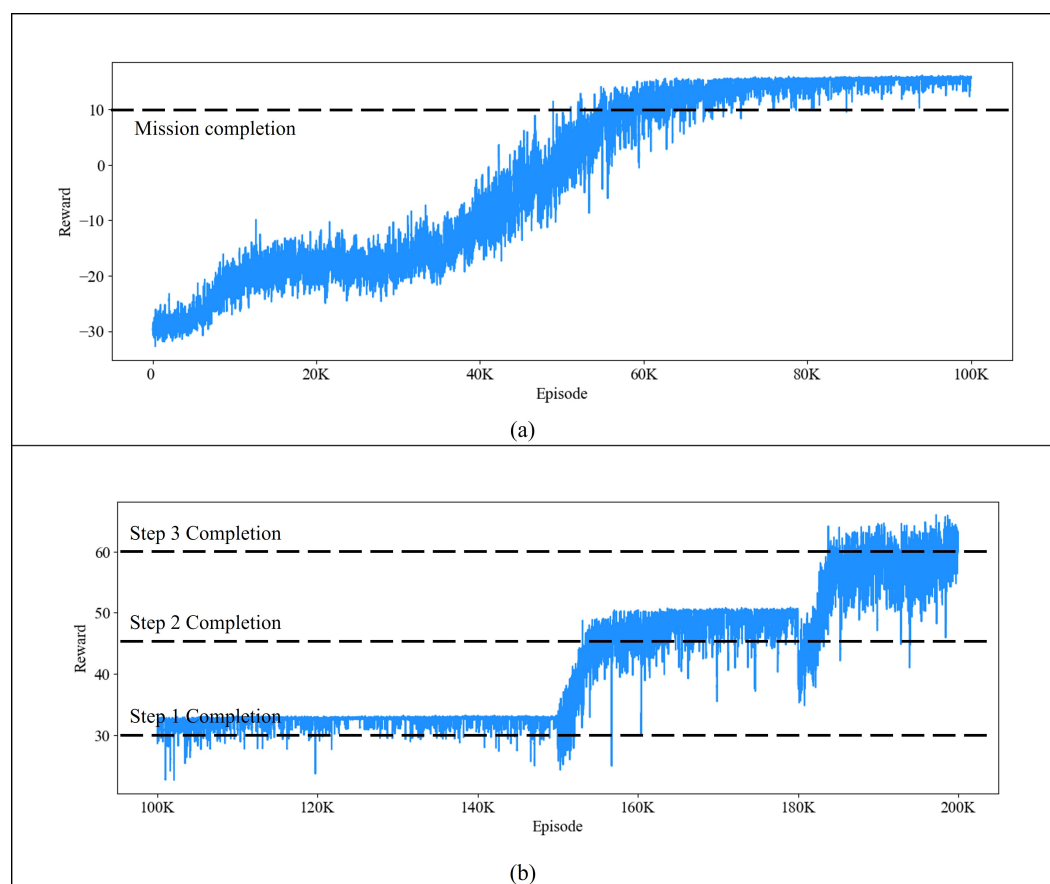


Figure 10. The learning curve in the training environment. (a) The learning curve during the first training phase. Upon reaching a total of 60,000 episodes, the UAV agent accomplished its subgoals. (b) The learning curve of the second training phase. The UAV agent started accomplishing all four subgoals when the episode reached 185,000.

6.2. Performance Evaluation in Test Scenarios

Figure 11 shows the trajectories followed by the UAV agent in the first scenario of the test phase. Goal-conditioned RL enables the agent to effectively accomplish the final goal by learning various maneuvers through achieving numerous subgoals, even if the subgoals were not encountered during training. In all cases, two types of agents trained in the first and second training stages achieved every subgoal with a minimum probability of 80% and 95%, respectively. It is possible for the agent, trained in only the first phase, to reach the subgoals and target point in the first scenario. In other words, the agent is limited in

carrying out more complex missions, like the second scenario, because it was only taught to move forward in the first phase.

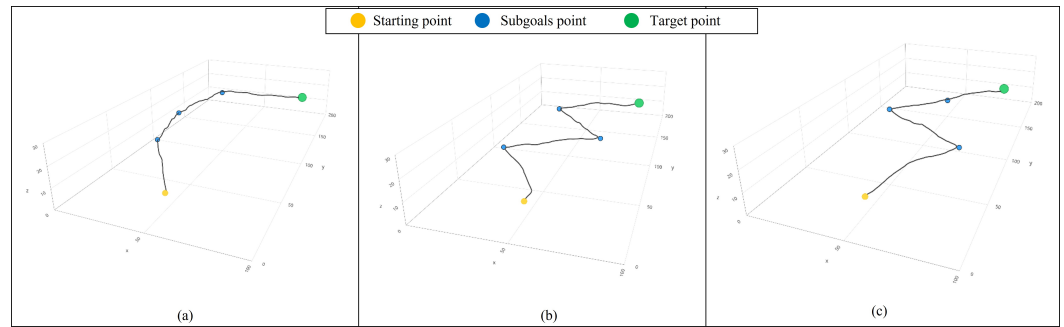


Figure 11. The simulation results in the first scenario for the UAV agent. (a–c) The results show three representative cases from 30 test trials where the agent successfully completed tasks while reaching the final destination with different subgoal configurations.

We utilized the agent trained only in the first stage of the proposed method to perform the tasks in the second scenario. As shown in Figure 12, the agent trained without curriculum learning cannot achieve goals in the second scenario. The agent without curriculum learning failed to reach the goal, requiring it to turn more than 90 degrees. Figure 12a shows that the agent failed to turn right after it passed the second subgoal and simply tried to move forward. Figure 12b shows that the agent attempted to turn left after the third subgoal, but failed and moved forward. As a result, an agent trained using only the first training stage cannot perform difficult tasks in the second scenario.

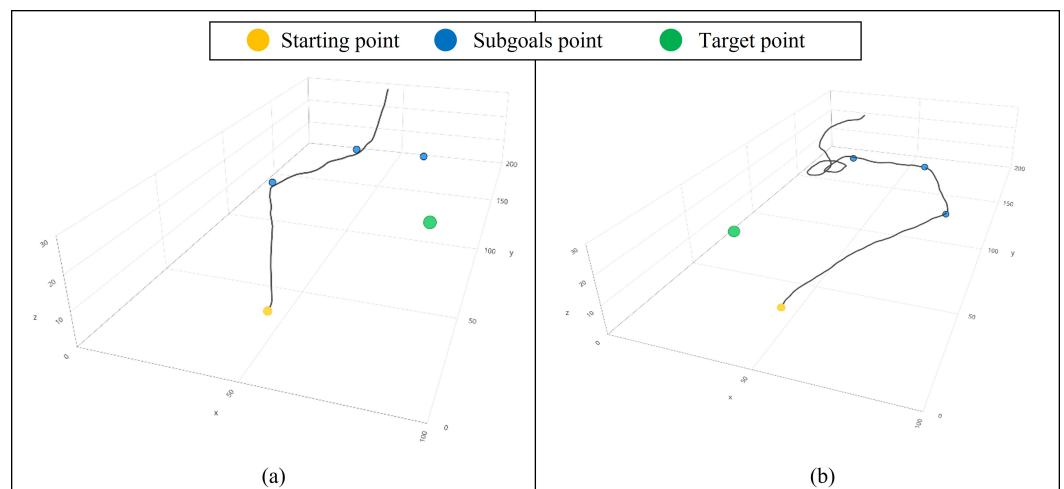


Figure 12. The simulation results in the second scenario for the UAV agent without curriculum learning. Two representative failure cases from 30 test trials are shown, where the agent failed to turn right after passing the second subgoal (a) and attempted but failed to turn left after the third subgoal (b).

The trajectories followed by the UAV agent utilizing the suggested method in the second test phase scenario are presented in Figure 13. The agent reached the final goal and passed the subgoals successfully. With curriculum learning, the agent could perform complex behaviors like round-trips and had a high success probability in all cases. In addition, it is notable that agents successfully perform more challenging missions than a round-trip. Figure 13c,d show that the agent can perform tasks that involve moving in multiple directions. The agent successfully performed two sharp directional changes and reached the final goal. The result of the second test phase demonstrates that through our

proposed method the UAV agent can perform a variety of tasks without additional training by task.

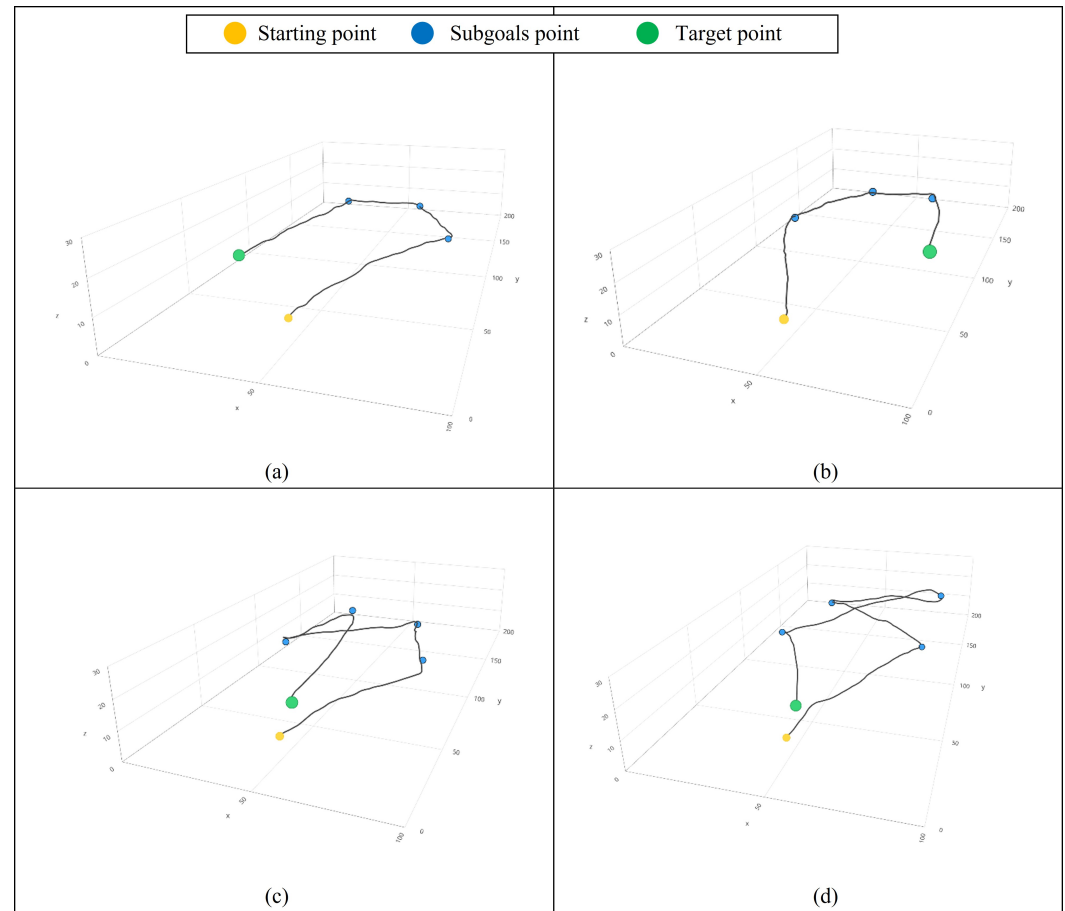


Figure 13. The simulation results in the second scenario for the UAV agent with our proposed method. Four different test cases from 30 trials demonstrate successful round-trip missions of varying complexity, including (a) clockwise rotation, (b) counter-clockwise rotation, (c,d) missions requiring directional changes.

The bar chart shown in Figure 14 shows the percentage of the agent reaching the subgoal by stage of curriculum learning in the second test scenario. The first stage indicates the baseline model with no curriculum learning. To evaluate the agent's likelihood of achieving subgoals, we considered it successful if the distance between the agent and the subgoal was less than 2 km. We established the criterion for success as the agent achieving the final goal after completing all subgoals. The success ratio for each subgoal the agent achieved in 30 tries was then calculated. The agent trained without curriculum learning does not complete the mission in all cases. Even when the agent performs relatively easy tasks in the second scenario, such as cases (a) and (b), it fails to succeed.

However, as the stage passes, the rate at which the agent accomplishes the entire scenario increases. The agent's ability to perform complex cases increases rapidly through curriculum learning. After the final stage, the agent achieves more than 70% of all cases of the second scenario. Because cases (c) and (d) were more complex than the previous two cases, these two cases had the least number of times the agent achieved all subgoals compared to other cases. Nevertheless, the agent successfully executed complex maneuvers like cases (c) and (d) with our proposed method.

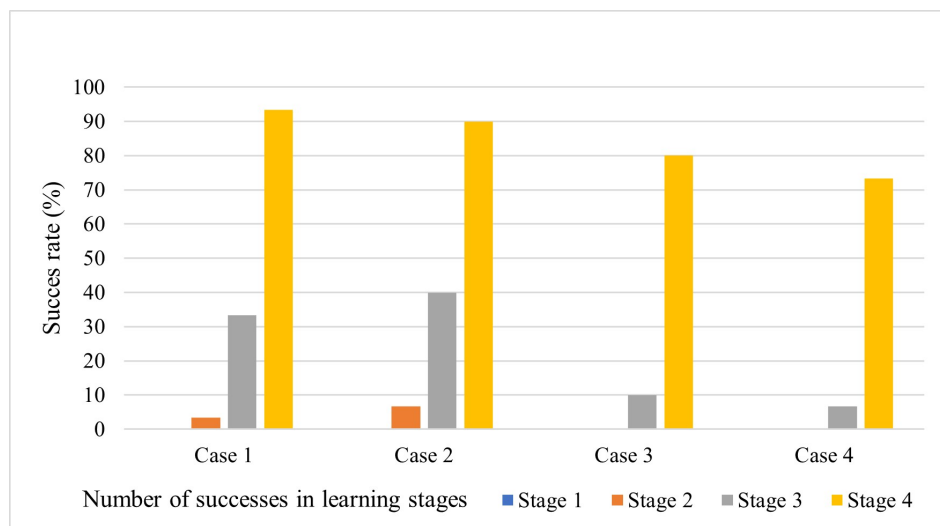


Figure 14. The plot illustrates the success rate in achieving the final goal in the second test scenario by the curriculum learning stage. A blue bar shows that, despite 30 attempts, the agent without curricular learning has never succeeded in reaching the final goal. In case 1, an orange bar shows that all subgoals have been met with a probability of 3.3% by the agent using the first curriculum learning phase.

Figure 15 depicts the ratio of the fully trained agent with our proposed method reaching subgoals in the second test scenario. It was observed that the probability of reaching subgoals decreased as the number of subgoals to be reached increased. In addition, cases (c) and (d), which are more difficult, have a relatively low probability of success. With our proposed method, we can observe that the agent succeeds in the turning maneuver with a probability of more than 90% regardless of the number of subgoals. Moreover, the agent successfully performs a complex maneuver requiring it to simultaneously perform clockwise and counterclockwise movements, such as cases (c) and (d).

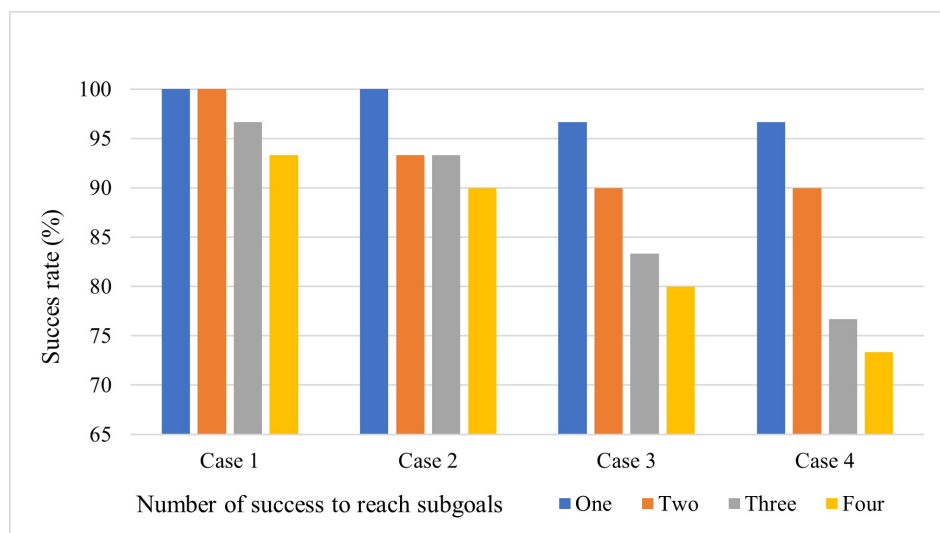


Figure 15. The plot illustrates the success rate of arrival at the subgoal using our proposed method during the second test scenario. In case 1, the agent has consistently accomplished more than one subgoal in 30 attempts, as indicated by a blue bar. In case 1, a yellow bar shows that, with a probability of 93.3%, the agent has achieved every subgoal.

As shown in Table 1, our method significantly outperforms the goal-conditioned only approach [1] across all scenarios. Particularly noteworthy is the improvement in complex tasks (Case 1 & 2) from 0% to 92% success rate, and the achievement of 77% success rate

in round-trip missions (Case 3 & 4) which were previously unattainable. As shown in Figures 14 and 15, mission complexity significantly affects performance, with success rates decreasing as the number of required subgoals increases:

- Single-goal missions: $\geq 95\%$ success rate
- Two-subgoal missions: $\geq 90\%$ success rate
- Three-subgoal missions: $\geq 75\%$ success rate
- Complex round-trip missions: $\geq 73\%$ success rate

Table 1. Comparison of Success Rates Across Different Methods.

Method	Simple Tasks	Complex Tasks (Case 1 & 2)	Round Trip (Case 3 & 4)
Goal-Conditioned Only [1]	80%	0%	0%
Our Method	95%	92%	77%

7. Discussion

7.1. Performance Analysis and Limitations

Compared to previous approach [1] that failed to achieve any success in complex tasks and round-trip missions (0% success rate), our method demonstrates remarkable improvements with 92% success rate in relatively complex tasks (Case 1 & 2) and 77% success rate in round-trip missions (Case 3 & 4). While existing methods were limited to simple forward navigation tasks, our approach successfully handles complex scenarios requiring multiple direction changes and round-trip capabilities. The performance advantage is particularly evident in tasks requiring sharp turns (>90 degrees), where our curriculum learning approach progressively improves performance from 0% to over 90% success rate. Moreover, our method shows robust performance with multiple subgoals, maintaining over 90% success rate for two-subgoal missions and 75% success rate for three-subgoal missions.

However, we identified some operational limitations through additional experiments. The agent could not achieve the final target and subgoals when trained without curriculum learning, as shown in Figure 14. While agents who completed our full training approach effectively achieved complex goals, we observed some limitations. Figure 16 shows cases where the agent passed by some subgoals and made unnecessary turns when reaching the target point. As shown in Figure 15, the performance notably degrades when dealing with multiple subgoals, particularly in scenarios requiring sharp turns or complex maneuvers. This suggests our approach might benefit from more sophisticated state representation and reward mechanisms.

Additionally, while achieving diverse missions successfully, the agent does not always take the most efficient route to the goal. This efficiency limitation becomes more pronounced in complex missions involving multiple waypoints or round-trip scenarios. Compared to other studies [38], our method prioritizes mission completion over path optimization. This performance-efficiency tradeoff reflects current limitations in simultaneously achieving high success rates and optimal path planning.

The significant overall performance gains can be attributed to our integration of curriculum learning with goal-conditioned RL, which enables systematic development of complex navigation capabilities. However, addressing the identified limitations in path efficiency and subgoal optimization remains an important direction for future work.

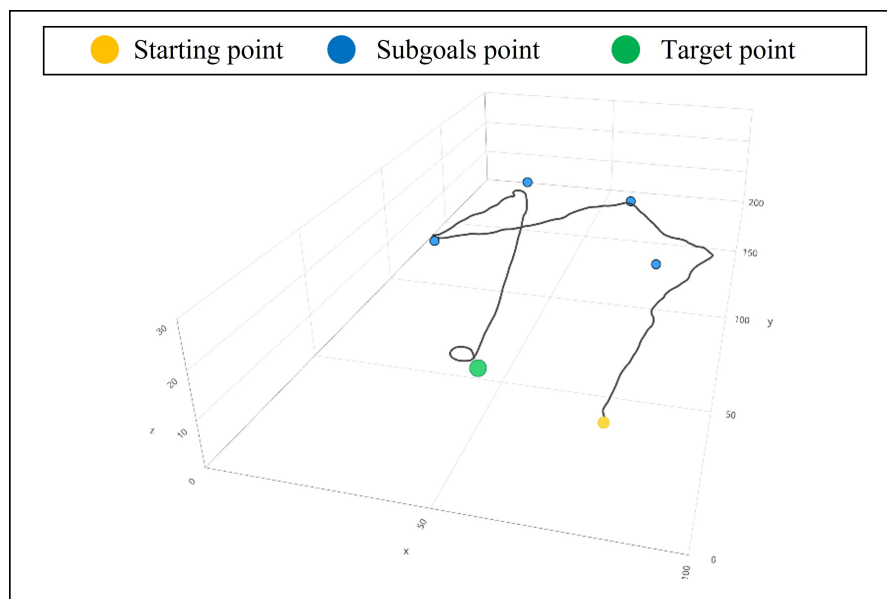


Figure 16. The simulation result in the second scenario for the UAV agent with our proposed method. Although the agent did not succeed in reaching the first subgoal, it achieved the final goal.

7.2. Implementation Challenges and Technical Limitations

In real-world environments, unexpected situations can arise, making it difficult for an agent to adapt to a variety of environments and perform various missions. While our method shows promise, several important limitations should be considered. Our current implementation uses a simplified 3 DoF model that does not account for crucial aerodynamic factors such as lift forces and stall conditions. Additionally, the simulation environment does not include real-world challenges such as wind disturbances, dynamic obstacles, or GPS inaccuracies, which could significantly impact actual flight performance. These environmental factors would need to be addressed for practical deployment.

The computational aspects of our approach also present certain challenges. The training process requires significant computational resources, taking approximately 4 h for the first phase and 6 h for the second phase. This computational burden increases with mission complexity and the number of subgoals. Furthermore, our discrete action space of 27 possible actions, while functional, may limit the smoothness of control compared to a continuous action space approach [39], particularly when precise adjustments are needed in real-world flight scenarios.

7.3. Future Directions and Applications

Future work could address these limitations by incorporating more realistic environmental factors, developing more efficient training algorithms, and exploring continuous action spaces for smoother control. Additionally, integrating obstacle avoidance capabilities and improving the efficiency of path planning while maintaining mission success rates would enhance the practical applicability of our approach. Despite these constraints, our method demonstrates significant potential for handling complex UAV missions through its unique combination of curriculum learning and goal-conditioned reinforcement learning.

8. Conclusions

The issue of determining the path for UAVs poses a crucial challenge for agents engaged in varying missions. Significantly, if an agent is trained to achieve a certain goal, its effectiveness may decline when employed in other situations. This paper presents a UAV agent that is fully controllable, utilizing goal-conditioned RL and curriculum learning

to solve these challenges. The subgoals effectively control the agent, enabling it to execute complex assignments in various settings successfully. The agent is adaptable and requires no additional training for deployment in various scenarios.

We conducted numerous experiments to verify the effectiveness of our suggested methodology. The empirical results show the efficacy of curriculum learning. Therefore, utilizing the proposed method, we can effectively manipulate the UAV agent by directing it toward multiple subgoals. Notably, the agent employed in each scenario is identical, and throughout training, the agent did not visit either the final objective or the subgoals. In addition, the UAV agent can execute intricate missions, such as making turns and completing round-trips.

However, the current study was only tested in an environment where the agent was unobstructed by obstacles. Therefore, it is necessary to introduce a learning technique that considers obstacles. Because the current state does not contain obstacle information, changing the components of the state and various experiments are required to make the agent recognize the obstacle independently.

Although the agent could perform various maneuvers fully, the agent did not choose the best action for the path to accomplish the subgoal. Furthermore, the agent sometimes made unnecessary movements when passing through subgoals. Moreover, there were some cases where subgoals were skipped. Consequently, further research is needed to enable the agent to acquire the optimal behavior to achieve all subgoals perfectly. We believe that our proposed method has a wide range of potential applications in path planning and can lead to future research.

Author Contributions: Conceptualization, H.K.; methodology, H.K. and J.C.; software, H.K. and J.C.; validation, J.C. and H.K.; formal analysis, J.C.; investigation, H.K.; resources, H.K.; data curation, H.K.; writing—original draft preparation, H.K.; writing—review and editing, J.C., H.D. and G.T.L.; visualization, H.K.; supervision, G.T.L. and H.D.; project administration, G.T.L.; funding acquisition, H.D. and G.T.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Lee, G.T.; Kim, C.O. Autonomous control of combat unmanned aerial vehicles to evade surface-to-air missiles using deep reinforcement learning. *IEEE Access* **2020**, *8*, 226724–226736. [\[CrossRef\]](#)
2. Shakhathreh, H.; Sawalmeh, A.H.; Al-Fuqaha, A.; Dou, Z.; Almaita, E.; Khalil, I.; Othman, N.S.; Khreishah, A.; Guizani, M. Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges. *IEEE Access* **2019**, *7*, 48572–48634. [\[CrossRef\]](#)
3. Yan, C.; Xiang, X.; Wang, C. Towards real-time path planning through deep reinforcement learning for a UAV in dynamic environments. *J. Intell. Robot. Syst.* **2020**, *98*, 297–309. [\[CrossRef\]](#)
4. Cui, Z.; Wang, Y. UAV path planning based on multi-layer reinforcement learning technique. *IEEE Access* **2021**, *9*, 59486–59497. [\[CrossRef\]](#)
5. Chen, X.; Chen, X.M.; Zhang, J. The dynamic path planning of UAV based on A* algorithm. *Appl. Mech. Mater.* **2014**, *494*, 1094–1097. [\[CrossRef\]](#)
6. Li, J.; Huang, Y.; Xu, Z.; Wang, J.; Chen, M. Path planning of UAV based on hierarchical genetic algorithm with optimized search region. In Proceedings of the 2017 13th IEEE International Conference on Control & Automation (ICCA), Ohrid, Macedonia, 3–6 July 2017; pp. 1033–1038.
7. Huang, C.; Fei, J. UAV path planning based on particle swarm optimization with global best path competition. *Int. J. Pattern Recognit. Artif. Intell.* **2018**, *32*, 1859008. [\[CrossRef\]](#)
8. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-level control through deep reinforcement learning. *Nature* **2015**, *518*, 529–533. [\[CrossRef\]](#) [\[PubMed\]](#)

9. Sinha, S.; Mandlekar, A.; Garg, A. S4rl: Surprisingly simple self-supervision for offline reinforcement learning in robotics. In Proceedings of the Conference on Robot Learning, PMLR, Auckland, New Zealand, 14–18 December 2022; pp. 907–917.
10. Hwang, H.J.; Jang, J.; Choi, J.; Bae, J.H.; Kim, S.H.; Kim, C.O. Stepwise Soft Actor–Critic for UAV Autonomous Flight Control. *Drones* **2023**, *7*, 549. [\[CrossRef\]](#)
11. Song, Y.; Romero, A.; Müller, M.; Koltun, V.; Scaramuzza, D. Reaching the limit in autonomous racing: Optimal control versus reinforcement learning. *Sci. Robot.* **2023**, *8*, eadg1462. [\[CrossRef\]](#) [\[PubMed\]](#)
12. Ma, B.; Liu, Z.; Dang, Q.; Zhao, W.; Wang, J.; Cheng, Y.; Yuan, Z. Deep reinforcement learning of UAV tracking control under wind disturbances environments. *IEEE Trans. Instrum. Meas.* **2023**, *72*, 2510913. [\[CrossRef\]](#)
13. Ma, B.; Liu, Z.; Zhao, W.; Yuan, J.; Long, H.; Wang, X.; Yuan, Z. Target tracking control of UAV through deep reinforcement learning. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 5983–6000. [\[CrossRef\]](#)
14. Wang, Y.; Boyle, D. Constrained reinforcement learning using distributional representation for trustworthy quadrotor UAV tracking control. *IEEE Trans. Autom. Sci. Eng.* **2024**. [\[CrossRef\]](#)
15. Choi, J.; Kim, H.M.; Hwang, H.J.; Kim, Y.D.; Kim, C.O. Modular Reinforcement Learning for Autonomous UAV Flight Control. *Drones* **2023**, *7*, 418. [\[CrossRef\]](#)
16. Qu, C.; Gai, W.; Zhong, M.; Zhang, J. A novel reinforcement learning based grey wolf optimizer algorithm for unmanned aerial vehicles (UAVs) path planning. *Appl. Soft Comput.* **2020**, *89*, 106099. [\[CrossRef\]](#)
17. Bouhamed, O.; Ghazzai, H.; Besbes, H.; Massoud, Y. Autonomous UAV navigation: A DDPG-based deep reinforcement learning approach. In Proceedings of the 2020 IEEE International Symposium on Circuits and Systems (ISCAS), Seville, Spain, 12–14 October 2020; pp. 1–5.
18. Luo, Y.; Ji, T.; Sun, F.; Liu, H.; Zhang, J.; Jing, M.; Huang, W. Goal-Conditioned Hierarchical Reinforcement Learning With High-Level Model Approximation. *IEEE Trans. Neural Netw. Learn. Syst.* **2024**. [\[CrossRef\]](#) [\[PubMed\]](#)
19. Ashraf, M.; Gaydamaka, A.; Tan, B.; Moltchanov, D.; Koucheryavy, Y. Low Complexity Algorithms for Mission Completion Time Minimization in UAV-Based Emergency Response. *IEEE Trans. Intell. Veh.* **2024**. [\[CrossRef\]](#)
20. Yang, R.; Lu, Y.; Li, W.; Sun, H.; Fang, M.; Du, Y.; Li, X.; Han, L.; Zhang, C. Rethinking goal-conditioned supervised learning and its connection to offline rl. *arXiv* **2022**, arXiv:2202.04478.
21. Zhao, R.; Sun, X.; Tresp, V. Maximum Entropy-Regularized Multi-Goal Reinforcement Learning. In Proceedings of the 36th International Conference on Machine Learning, Long Beach, CA, USA, 9–15 June 2019; Volume 97, Proceedings of Machine Learning Research, pp. 7553–7562.
22. Nasiriany, S.; Pong, V.; Lin, S.; Levine, S. Planning with goal-conditioned policies. *Adv. Neural Inf. Process. Syst.* **2019**, *32*.
23. Lee, G.T.; Kim, K. A Controllable Agent by Subgoals in Path Planning Using Goal-Conditioned Reinforcement Learning. *IEEE Access* **2023**, *11*, 33812–33825. [\[CrossRef\]](#)
24. Bhatnagar, S.; Sutton, R.S.; Ghavamzadeh, M.; Lee, M. Natural actor–critic algorithms. *Automatica* **2009**, *45*, 2471–2482. [\[CrossRef\]](#)
25. Burda, Y.; Edwards, H.; Pathak, D.; Storkey, A.; Darrell, T.; Efros, A.A. Large-scale study of curiosity-driven learning. *arXiv* **2018**, arXiv:1808.04355.
26. Pathak, D.; Agrawal, P.; Efros, A.A.; Darrell, T. Curiosity-driven exploration by self-supervised prediction. In Proceedings of the International Conference on Machine Learning, PMLR, Sydney, Australia, 6–11 August 2017; pp. 2778–2787.
27. Burda, Y.; Edwards, H.; Storkey, A.; Klimov, O. Exploration by random network distillation. *arXiv* **2018**, arXiv:1810.12894.
28. Mnih, V.; Badia, A.P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In Proceedings of the International Conference on Machine Learning, PMLR, New York, NY, USA, 19–24 June 2016; pp. 1928–1937.
29. Oh, J.; Guo, Y.; Singh, S.; Lee, H. Self-imitation learning. In Proceedings of the International Conference on Machine Learning, PMLR, Stockholm, Sweden, 10–15 July 2018; pp. 3878–3887.
30. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal policy optimization algorithms. *arXiv* **2017**, arXiv:1707.06347.
31. Bengio, Y.; Louradour, J.; Collobert, R.; Weston, J. Curriculum learning. In Proceedings of the 26th Annual International Conference on Machine Learning, Montreal, QC, Canada, 14–18 June 2009; pp. 41–48.
32. Ivanovic, B.; Harrison, J.; Sharma, A.; Chen, M.; Pavone, M. Barc: Backward reachability curriculum for robotic reinforcement learning. In Proceedings of the 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019; pp. 15–21.
33. Silva, F.L.D.; Costa, A.H.R. Object-oriented curriculum generation for reinforcement learning. In Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems, Stockholm, Sweden, 10–15 July 2018; pp. 1026–1034.
34. Narvekar, S.; Peng, B.; Leonetti, M.; Sinapov, J.; Taylor, M.E.; Stone, P. Curriculum learning for reinforcement learning domains: A framework and survey. *J. Mach. Learn. Res.* **2020**, *21*, 7382–7431.

35. Kim, S.; Kim, Y. Three dimensional optimum controller for multiple UAV formation flight using behavior-based decentralized approach. In Proceedings of the 2007 International Conference on Control, Automation and Systems, Seoul, Republic of Korea, 17–20 October 2007; pp. 1387–1392.
36. Lee, G.; Kim, K.; Jang, J. Real-time path planning of controllable UAV by subgoals using goal-conditioned reinforcement learning. *Appl. Soft Comput.* **2023**, *146*, 110660. [[CrossRef](#)]
37. Konda, V.; Tsitsiklis, J. Actor-critic algorithms. *Adv. Neural Inf. Process. Syst.* **1999**, *12*, 1008–1014.
38. Razzaghi, P.; Tabrizian, A.; Guo, W.; Chen, S.; Taye, A.; Thompson, E.; Bregeon, A.; Baheri, A.; Wei, P. A survey on reinforcement learning in aviation applications. *Eng. Appl. Artif. Intell.* **2024**, *136*, 108911. [[CrossRef](#)]
39. Sanz, D.; Valente, J.; del Cerro, J.; Colorado, J.; Barrientos, A. Safe operation of mini UAVs: A review of regulation and best practices. *Adv. Robot.* **2015**, *29*, 1221–1233. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.